Computed-Torque Controller & Tendon Dynamics

Direct-Drive Joint 1 — Cable-Spring SEA Joint 2

Isaac-sim robotics notes

March 2026

Contents

1 File Map

File	Role
`robots/cup_manipulator_tendon.py`	`ComputedTorqueController` (Drake `LeafSystem`)
`test_cup_manipulator_pendulam_with_spring_damper_pydrake_v2.py`	`TendonSpringCompliance` (SEA leaf system), `ComputedTorqueSimulation` (diagram wiring)
`cable.py`	`PulleyBase`, pulley geometry, pitch-radius constant

2 Robot Overview: 2-DOF Planar Manipulator

The cup manipulator has two revolute joints in the horizontal plane:

Joint	URDF name	Transmission
q_1	`link1_base`	Direct-drive (no pulley in the torque path)
q_2	`link2_link1`	Cable & HTD-5M belt via pulley of radius r_p

Link lengths (URDF geometry):

$$L_1 = 342.47\,\text{mm}, \qquad L_2 = 190.00\,\text{mm}.$$

HTD-5M 60-tooth pulley pitch radius:

$$r_p = \frac{N \cdot p}{2\pi} = \frac{60 \times 5\,\text{mm}}{2\pi} \approx 47.75\,\text{mm}. \tag{1}$$

3 Rigid-Body Dynamics (Plant)

Drake models both joints as a single `MultibodyPlant`. The standard manipulator equation is:

$$M(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + g(\boldsymbol{q}) = \boldsymbol{\tau}_{\text{plant}} \tag{2}$$

where $\boldsymbol{q} = [q_1, q_2]^\top$, M is the 2×2 mass matrix, $C\dot{\boldsymbol{q}}$ collects Coriolis and centrifugal terms, and g is the gravity vector. $\boldsymbol{\tau}_{\text{plant}}$ is the torque vector that arrives at the plant's actuation input port from either the controller (rigid tendon) or the SEA spring (cable-spring mode).

```
# Drake plant setup for both joints
plant = MultibodyPlant(time_step=0.002) # dt = 2 ms
parser = Parser(plant, scene_graph)
manipulator.load_urdf_to_plant(plant, parser)
manipulator.weld_base_to_world(plant)
manipulator.add_end_effector_frame(plant)
manipulator.set_joint_properties(plant) # viscous damping
manipulator.add_joint_actuators(plant) # tau_link1_base, tau_link2_link1
plant.Finalize()
```

4 Joint 1: Direct-Drive Dynamics

Joint 1 is mechanically coupled directly to the motor shaft (no belt). Newton's 2nd law on the motor+link1 inertia:

$$\tau_1 = \tau_{1,\text{cmd}} \tag{3}$$

The commanded torque reaches the joint without modification. Drake's actuator for joint 1 is:

```
# robots/cup_manipulator_tendon.py -- add_joint_actuators()
jt1 = manipulator.get_joint_by_name(plant, "link1_base")
act1 = plant.AddJointActuator("tau_link1_base", jt1, effort_limit)
# Optional motor model (AK80_8_KV60_Config etc.)
act1.set_default_rotor_inertia(rotor_inertia)
act1.set_default_gear_ratio(gear_ratio)
```

When the motor model is configured, Drake automatically reflects the rotor inertia and gear ratio into the plant dynamics, so $M(\boldsymbol{q})$ is augmented by $N^2 I_{\text{rotor}}$ on the diagonal entry for q_1.

5 Joint 2: Cable & Spring (SEA) Dynamics

5.1 Rigid-Tendon Baseline (SEA Disabled)

When $k_s = 0$ (default), the cable is modelled as perfectly rigid. The commanded torque is delivered directly:

$$\tau_{2,\text{plant}} = \tau_{2,\text{cmd}}. \tag{4}$$

5.2 Series Elastic Actuator (SEA) Model

When $k_s > 0$ a physical spring is inserted in the cable between the motor pulley and joint 2. The motor angle θ_m and joint angle q_2 are now *separate* degrees of freedom connected by the spring.

Cable stretch

$$\delta = r_p\,(\theta_m - q_2) \qquad [\text{m}] \tag{5}$$

Spring (cable) force

$$F_s = k_s\,\delta + b_s\,\dot{\delta} = k_s\,r_p\,(\theta_m - q_2) + b_s\,r_p\,(\dot{\theta}_m - \dot{q}_2) \qquad [\text{N}] \tag{6}$$

Torque on joint 2 from spring

$$\tau_{2,\text{plant}} = F_s\,r_p = k_s\,r_p^2\,(\theta_m - q_2) + b_s\,r_p^2\,(\dot{\theta}_m - \dot{q}_2) \qquad [\text{Nm}] \tag{7}$$

Motor equation of motion Newton's 2nd law on the motor shaft (inertia I_m):

$$I_m\,\ddot{\theta}_m = \tau_{2,\text{cmd}} - F_s\,r_p = \tau_{2,\text{cmd}} - k_s\,r_p^2\,(\theta_m - q_2) - b_s\,r_p^2\,(\dot{\theta}_m - \dot{q}_2) \tag{8}$$

Full augmented system state Adding the SEA to the plant extends the state from $\boldsymbol{x}_{\text{plant}} = [q_1, q_2, \dot{q}_1, \dot{q}_2]^\top$ to

$$\boldsymbol{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \\ \theta_m \\ \dot{\theta}_m \end{bmatrix}, \quad \dot{\boldsymbol{x}} = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ M^{-1}(\boldsymbol{\tau}_{\text{plant}} - C\dot{\boldsymbol{q}} - g) \\ \dot{\theta}_m \\ \frac{1}{I_m}\left(\tau_{2,\text{cmd}} - F_s\, r_p\right) \end{bmatrix} \tag{9}$$

where $\boldsymbol{\tau}_{\text{plant}} = [\tau_{1,\text{cmd}},\ F_s\, r_p]^\top$.

Physical intuition. A weak spring ($k_s \approx 50$–$200\,\text{N/m}$) stretches significantly under load: θ_m advances while q_2 lags. This creates oscillatory, sluggish joint-2 behaviour reminiscent of a tendon with finite stiffness. A stiff spring ($k_s \gtrsim 5000\,\text{N/m}$) makes $\theta_m \approx q_2$, recovering the rigid-tendon baseline.

5.3 TendonSpringCompliance LeafSystem

The SEA is implemented as a Drake `LeafSystem` with two continuous states $[\theta_m, \dot{\theta}_m]$:

```
# test_cup_manipulator_pendulam_with_spring_damper_pydrake_v2.py

class TendonSpringCompliance(LeafSystem):
    # Ports
    # In: torque_cmd [2] -- [tau1_cmd, tau2_motor_cmd] from controller [Nm]
    # In: plant_state [n] -- read q2, qdot2 from plant
    # Out: torque_out [2] -- [tau1, tau2_spring] to plant actuation port [Nm]
    # Out: spring_force[1] -- F_s [N] (logged separately)
    # Continuous state: [theta_m, theta_m_dot]

    def __init__(self, plant, manipulator,
                 k_s=500.0, b_s=2.0, I_m=5e-4):
        super().__init__()
        self._r_p = manipulator.PULLEY_RADIUS # ~0.04775 m
        self._k_s = k_s; self._b_s = b_s; self._I_m = I_m
        self._plant_ctx = plant.CreateDefaultContext() # private, for q2 reads
        nstate = plant.num_multibody_states()
        self._cmd_port = self.DeclareVectorInputPort("torque_cmd", 2)
        self._state_port = self.DeclareVectorInputPort("plant_state", nstate)
        self.DeclareVectorOutputPort("torque_out", 2, self._calc_output)
        self.DeclareVectorOutputPort("spring_force", 1, self._calc_spring_force)
        self.DeclareContinuousState(2) # [theta_m, theta_m_dot] continuous states
```

5.3.1 Time Derivatives (Motor ODE)

```
    def _spring_force(self, context):
        # F_s = k_s * delta+ b_s * delta_dot [N]
        x = context.get_continuous_state_vector().CopyToVector()
        theta_m, theta_m_dot = x[0], x[1]
        q2, q2dot = self._read_joint2(context) # from plant_state port
        r_p = self._r_p
        delta= r_p * (theta_m - q2) # cable stretch [m]
        delta_dot = r_p * (theta_m_dot - q2dot) # rate [m/s]
        return self._k_s * delta+ self._b_s * delta_dot # [N]

    def DoCalcTimeDerivatives(self, context, derivatives):
        tau_cmd = self._cmd_port.Eval(context)
```

```
        x = context.get_continuous_state_vector().CopyToVector()
        theta_m_dot = x[1]
        F_s = self._spring_force(context)
        # Motor EOM: I_m * theta_m_ddot = tau2_cmd - F_s * r_p
        theta_m_ddot = (tau_cmd[1] - F_s * self._r_p) / self._I_m
        derivatives.get_mutable_vector().SetAtIndex(0, theta_m_dot)
        derivatives.get_mutable_vector().SetAtIndex(1, theta_m_ddot)
```

5.3.2 Output: Torque Delivered to Plant

```
    def _calc_output(self, context, output):
        tau_cmd = self._cmd_port.Eval(context)
        tau2_spring = self._spring_force(context) * self._r_p # [Nm]
        # Joint 1: tau1 passes unmodified (direct-drive, no spring)
        # Joint 2: spring torque F_s*r_p delivered, NOT tau2_cmd directly
        output.SetFromVector(np.array([tau_cmd[0], tau2_spring]))
```

5.3.3 Initialisation: Zero Spring Stretch at $t = 0$

To avoid an impulsive jerk at startup, the motor angle is seeded to match the robot's initial joint-2 angle:

```
    # In ComputedTorqueSimulation.initialize()
    if self.tendon_sys is not None:
        tendon_ctx = self.tendon_sys.GetMyMutableContextFromRoot(
            self.simulator.get_mutable_context()
        )
        # theta_m = q2 -- no initial spring stretch
        tendon_ctx.get_mutable_continuous_state_vector().SetAtIndex(0, q0[1])
        # theta_m_dot = 0
        tendon_ctx.get_mutable_continuous_state_vector().SetAtIndex(1, 0.0)
```

6 Computed-Torque (Inverse-Dynamics) Controller

6.1 Feedback-Linearisation Law

The computed-torque law cancels the nonlinear plant dynamics and imposes desired linear 2nd-order error dynamics.

Step 1 — IK: Cartesian target → joint target

$$\boldsymbol{q}_{\text{des}}(t) = \text{IK}\big(\boldsymbol{p}_{\text{ee,des}}(t)\big) \tag{10}$$

Analytical 2R inverse kinematics; warm-started from the previous solution to remain on the same elbow branch.

Step 2 — Reference feedforward in joint space The 2R Jacobian at $\boldsymbol{q}_{\text{des}}$:

$$J(\boldsymbol{q}) = \begin{bmatrix} -L_1 \sin q_1 - L_2 \sin(q_1 + q_2) & -L_2 \sin(q_1 + q_2) \\ L_1 \cos q_1 + L_2 \cos(q_1 + q_2) & L_2 \cos(q_1 + q_2) \end{bmatrix} \tag{11}$$

Reference joint velocity and acceleration via pseudoinverse:

$$\dot{\boldsymbol{q}}_{\text{ref}} = J^{+} \dot{\boldsymbol{p}}_{\text{ref}}, \qquad \ddot{\boldsymbol{q}}_{\text{ref}} \approx J^{+} \ddot{\boldsymbol{p}}_{\text{ref}} \quad (\dot{J}\dot{\boldsymbol{q}} \text{ bias dropped}) \tag{12}$$

Step 3 — Desired joint acceleration (PD + feedforward)

$$\boxed{\boldsymbol{a}_{\text{des}} = \ddot{\boldsymbol{q}}_{\text{ref}} + K_p\,(\boldsymbol{q}_{\text{des}} - \boldsymbol{q}) + K_d\,(\dot{\boldsymbol{q}}_{\text{ref}} - \dot{\boldsymbol{q}})} \tag{13}$$

Step 4 — Inverse dynamics: acceleration → torque

$$\boldsymbol{\tau} = M(\boldsymbol{q})\,\boldsymbol{a}_{\text{des}} + C(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + g(\boldsymbol{q}) \tag{14}$$

Drake evaluates (**??**) via `CalcInverseDynamics` (internally uses the equation $\boldsymbol{\tau} = M\,\dot{v} + \text{bias}$, gravity included):

```
# robots/cup_manipulator_tendon.py -- ComputedTorqueController._solve()

# Map a_des from user-order [q1,q2] to Drake nv-vector
a_des_drake = np.zeros(nv)
a_des_drake[self._v_idx[0]] = a_des_user[0]
a_des_drake[self._v_idx[1]] = a_des_user[1]

# Inverse dynamics: tau= M(q)*a_des + C(q,v)*v + g(q)
self._forces.SetZero() # no additional external forces
tau_full = self._plant.CalcInverseDynamics(
    self._plant_ctx, a_des_drake, self._forces
)
tau1 = float(tau_full[self._v_idx[0]])
tau2 = float(tau_full[self._v_idx[1]])
```

6.2 Closed-Loop Error Dynamics

Substituting (**??**) into (**??**) gives decoupled linear 2nd-order systems for each joint:

$$\ddot{e}_i + K_d\,\dot{e}_i + K_p\,e_i = 0, \qquad e_i = q_{i,\text{des}} - q_i \tag{15}$$

with complex poles at:

$$s = -\zeta\omega_n \pm \omega_n\sqrt{\zeta^2 - 1}, \qquad \omega_n = \sqrt{K_p}, \quad \zeta = \frac{K_d}{2\sqrt{K_p}} \tag{16}$$

Default gains: $K_p = 400\,\text{s}^{-2}$, $K_d = 40\,\text{s}^{-1} \Rightarrow \omega_n = 20\,\text{rad/s}$, $\zeta = 1$ (critically damped).

```
# Feedforward + PD law -- ComputedTorqueController._solve()
ee_vel_ref = self._vel_port.Eval(context) # EE vel ref [m/s]
ee_acc_ref = self._acc_port.Eval(context) # EE acc ref [m/s^2]
q1d, q2d = q_des
s1 = np.sin(q1d); c1 = np.cos(q1d)
s12 = np.sin(q1d + q2d); c12 = np.cos(q1d + q2d)
J = np.array([
    [-self._L1 * s1 - self._L2 * s12, -self._L2 * s12],
    [ self._L1 * c1 + self._L2 * c12, self._L2 * c12],
])
J_inv = np.linalg.pinv(J)
q_dot_ref = J_inv @ ee_vel_ref # joint vel ref [rad/s]
q_ddot_ref = J_inv @ ee_acc_ref # joint acc ref [rad/s^2]

# Full CT: feedforward + PD feedback
a_des_user = (q_ddot_ref
              + self._Kp * (q_des - q)
              + self._Kd * (q_dot_ref - q_dot))
```

6.3 Joint-2 Cable Tension Decomposition

Because cables can only pull ($T \geq 0$), the signed torque τ_2 is resolved into two non-negative tensions (retracting and extending cables):

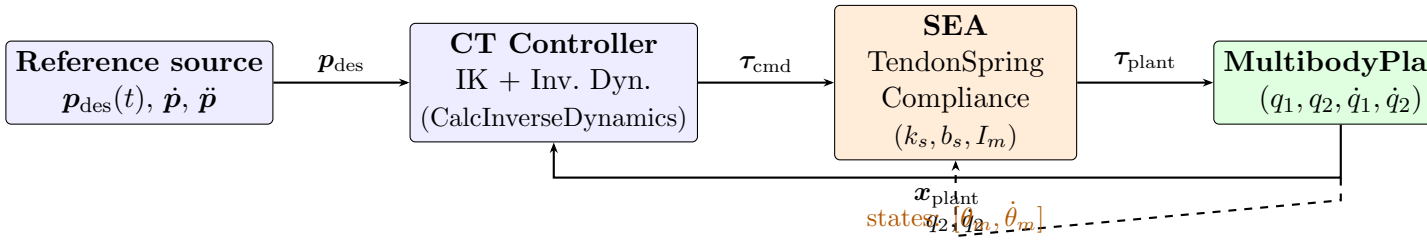
$$F_{\text{net}} = \frac{\tau_2}{r_p}, \qquad T_{\text{green}} = \max(F_{\text{net}},\ 0), \qquad T_{\text{red}} = \max(-F_{\text{net}},\ 0) \tag{17}$$

$$\tau_{2,\text{cmd}} = (T_{\text{green}} - T_{\text{red}})\, r_p = \tau_2 \tag{18}$$

```
# Cable tension decomposition -- ComputedTorqueController._solve()
F_net = tau2 / self._r_p # net cable force [N]
T_green = max(F_net, 0.0) # retracting side (>=0)
T_red = max(-F_net, 0.0) # extending side (>=0)
tau2_cmd = (T_green - T_red) * self._r_p # = tau2; logged separately
tau_clip = np.clip([tau1, tau2_cmd], -self._tau_max, self._tau_max)
```

7 Diagram: Controller + SEA Wiring

7.1 Block Diagram



Dashed arrow: the `plant_state` port is connected to the SEA to read $q_2, \dot{q}_2$ at each integration step. When the SEA is disabled ($k_s = 0$), the direct wire $\boldsymbol{\tau}_{\text{cmd}} \rightarrow$ plant is used instead.

7.2 Drake Diagram Wiring Code

```
# ComputedTorqueSimulation.connect_and_build()

# Controller receives: EE reference + plant state
builder.Connect(ee_ref.get_output_port(),
                ik_system.GetInputPort("desired_ee_pos"))
builder.Connect(ee_vel_src.get_output_port(),
                ik_system.GetInputPort("ee_vel_ref"))
builder.Connect(ee_acc_src.get_output_port(),
                ik_system.GetInputPort("ee_acc_ref"))
builder.Connect(plant.get_state_output_port(),
                ik_system.GetInputPort("plant_state"))

if tendon_stiffness > 0: # SEA enabled
    sea = builder.AddSystem(TendonSpringCompliance(
        plant, manipulator,
        k_s=tendon_stiffness,
        b_s=tendon_damping,
        I_m=tendon_motor_inertia,
    ))
    # Controller -- SEA -- plant
    builder.Connect(ik_system.GetOutputPort("actuation"),
                    sea.GetInputPort("torque_cmd"))
    builder.Connect(plant.get_state_output_port(),
```

```
                    sea.GetInputPort("plant_state")) # feeds q2, qdot2
    builder.Connect(sea.GetOutputPort("torque_out"),
                    plant.get_actuation_input_port())
else: # rigid tendon
    builder.Connect(ik_system.GetOutputPort("actuation"),
                    plant.get_actuation_input_port())
```

8 Gain Selection Guide

Parameter	Default	Interpretation / effect
K_p	400 s^{-2}	Natural frequency $\omega_n = \sqrt{K_p}$. Larger K_p $\rightarrow$ faster convergence; too large $\rightarrow$ instability with SEA.
K_d	40 s^{-1}	Damping ratio $\zeta = K_d/(2\omega_n)$. Default $\zeta = 1$ (critically damped, no overshoot).
k_s	0 N/m	SEA spring stiffness. $k_s = 0$ $\rightarrow$ rigid tendon. $k_s = 100$ $\rightarrow$ noticeably laggy J2. $k_s = 5000$ $\rightarrow$ quasi-rigid.
b_s	2 N·s/m	SEA cable viscous damping. Increases spring damping ratio.
I_m	5×10^{-4} kg·m^2	Motor rotor inertia. Larger I_m $\rightarrow$ longer oscillation period (slower motor response).

9 CLI Usage

```
# Rigid tendon (default)
python test_cup_manipulator_pendulam_with_spring_damper_pydrake_v2.py \
    --mode computed-torque --duration 10

# Weak SEA (slug-like J2, clear lag)
python ... --mode computed-torque \
    --tendon-stiffness 100 --tendon-damping 2 --tendon-motor-inertia 5e-4

# Stiff SEA (near-rigid, small residual oscillation)
python ... --mode computed-torque \
    --tendon-stiffness 5000 --tendon-damping 5 --tendon-motor-inertia 5e-4

# Custom CT gains (higher bandwidth)
python ... --mode computed-torque --ct-kp 900 --ct-kd 60
```